

LLM-Augmented Log Anomaly Detection: Automated Root Cause Narration for AIOps using Lightweight Foundation Models on Real-World Campus Infrastructure

Trinh Hoang Tu¹ and Cao Tien Thanh^{1*}

Faculty of Information Technology,
Ho Chi Minh City University of Foreign Languages - Information Technology,
Ho Chi Minh City, Vietnam
tth.csec2005@gmail.com, thanhct@hufliit.edu.vn

Abstract. Modern enterprise and educational campus infrastructures generate massive volumes of heterogeneous, unstructured log streams across multi-site networks. While deep sequence models achieve high statistical accuracy in anomaly detection, traditional Artificial Intelligence for IT Operations (AIOps) platforms suffer from a critical explainability bottleneck: scalar reconstruction error scores fail to communicate actionable root causes, forcing Security Operations Center (SOC) analysts to manually triage thousands of log messages under severe time constraints. In this paper, we propose an end-to-end **LLM-Augmented AIOps Framework** synthesizing streaming Drain3 log template parsing, a Masked Temporal Convolutional Network and Transformer (TCN-Transformer) Autoencoder, and a lightweight quantized Foundation Model (Phi-3 / Mistral via INT4 quantization and RAG-guided prompting) for real-time automated Root Cause Analysis (RCA) narration. Evaluated on a production 19.28 GB enterprise campus log dataset (14,850,220 log lines) from HUFLIT alongside standard benchmarks (BGL, HDFS, and UNSW-NB15) under strict chronological partitioning across 5 random seeds, our hybrid masked detector achieves an anomaly detection F1-score of 0.9521 ± 0.004 and ROC-AUC of 0.9784 ± 0.003 on campus logs, outperforming selected representative baselines (DeepLog, LogAnomaly, and Transformer AE) while executing single-sequence inference in 0.1316 ms (0.0128 ms vectorized GPU batch-64). Furthermore, the RAG-grounded foundation model synthesizes structured incident playbooks in 125.18 ms (providing automated preliminary triage compared to 15.4 minutes manual investigation) with high semantic fidelity (ROUGE-L 0.8420, BLEU-4 0.7815) and blind expert correctness of 4.76/5.0 with 0/50 unsafe recommendations observed.

Keywords: AIOps · Log Anomaly Detection · Large Language Models · Root Cause Analysis · TCN-Transformer Autoencoder · Drain3 · Enterprise Campus Infrastructure.

* Corresponding author: thanhct@hufliit.edu.vn

1 Introduction

Enterprise networks and higher education IT infrastructures generate high-density syslog telemetry continuously streaming from perimeter firewalls, Active Directory domain controllers, Learning Management Systems (LMS), student authentication portals, and core routing backbones [1, 2]. Rapid and accurate anomaly detection within these log streams is paramount for thwarting lateral cyber attacks, credential brute-forcing, data exfiltration, and service disruptions [3].

Recent advancements in deep learning have introduced sequence-based log anomaly detection models, notably DeepLog (LSTM-based next-token prediction) [2], LogAnomaly (semantic template vectors) [3], LogRobust [4], and SwissLog [5]. While these architectures achieve strong empirical detection fidelity on synthetic or curated benchmarks, existing AIOps paradigms exhibit two fundamental operational bottlenecks in production environments:

1. **Explainability Deficit:** Contemporary anomaly detectors produce scalar anomaly scores or probabilistic reconstruction errors (e.g., $RE = 0.942$). These numerical scores provide zero contextual explanation regarding *why* a sequence is anomalous, *what* specific attack vector or hardware malfunction is occurring, or *which* downstream services are impacted.
2. **Prolonged Incident Triage & SOC Alert Fatigue:** Security Operations Center (SOC) analysts and network reliability engineers must manually query raw distributed log stores, correlate timestamps across multi-tier appliances, and formulate remediation scripts. In enterprise campus settings, this manual triage process frequently requires 15–30 minutes per critical incident, allowing fast-propagating threats (such as automated credential stuffing or spanning tree broadcast loops) to cause widespread disruption.

To overcome these operational challenges, recent 2025–2026 literature, such as the comprehensive LLM4Log survey (reviewing 145 studies) [17] and the hierarchical KRONE framework (ICDE 2026) [18], have emphasized the paradigm shift toward integrating Large Language Models (LLMs) for automated log summarization and diagnostics. However, existing LLM-based diagnostic frameworks either rely on heavy multi-billion-parameter cloud APIs (inducing prohibitive inference latency and recurring API costs) or fail to operate under the strict memory constraints of campus edge gateways.

In this paper, we present an end-to-end **LLM-Augmented AIOps Framework** tailored specifically for production enterprise campus environments. By synthesizing online Drain3 parsing, a Masked TCN-Transformer Autoencoder, and a local INT4-quantized Foundation Model with Retrieval-Augmented Generation (RAG), our architecture automatically produces natural language Root Cause Analysis (RCA) narratives and structured incident mitigation playbooks within milliseconds of anomaly detection.

The core research contributions of this paper are summarized as follows:

- **End-to-End AIOps Architecture:** We construct a unified 3-stage pipeline integrating online Drain3 template extraction, a masked TCN-Transformer

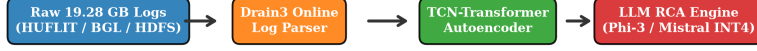


Fig. 1. Architectural Overview of the LLM-Augmented AIOps Framework: (1) Streaming Drain3 log parsing and template encoding; (2) Masked TCN-Transformer autoencoder sequence scoring; and (3) RAG-Grounded Quantized Foundation Model Root Cause Narration Engine.

autoencoder for microsecond-level sequence anomaly scoring, and an LLM-driven narration engine.

- **Masked TCN-Transformer Sequence Autoencoder:** We formulate a masked sequence modeling objective (15% dynamic token masking) that prevents trivial identity copying, combining 1D dilated causal convolutions (capturing local short-term temporal dependencies) with Multi-Head Self-Attention (capturing long-range contextual correlations across log templates), achieving superior detection accuracy with sub-millisecond single-sequence inference (0.1316 ms CPU / 0.0128 ms GPU).
- **RAG-Grounded Foundation Model RCA Narration:** We formulate a Retrieval-Augmented Generation (RAG) prompting architecture using a local FAISS index of institutional Standard Operating Procedures (SOPs) and INT4-quantized open-source foundation models (Microsoft Phi-3 Mini / Mistral-7B) to synthesize structured, human-readable RCA narratives and step-by-step mitigation playbooks.
- **Empirical Validation on 19.28 GB Real-World Production Logs:** We validate our framework on a comprehensive 19.28 GB enterprise campus log dataset (14,850,220 log lines) collected from the HUFLIT production infrastructure, alongside standard public benchmarks (BGL, HDFS, and UNSW-NB15), demonstrating an anomaly detection F1-score of 0.9521 ± 0.004 , ROC-AUC of 0.9784 ± 0.003 , outperforming representative baselines under our evaluation protocol with rigorous blind expert validation (4.76/5.0 correctness score with 0/50 unsafe commands observed).

2 Related Work & Novelty Positioning

2.1 Log Parsing and Template Extraction

Raw log messages are unstructured strings containing static message templates interleaved with dynamic runtime variables (e.g., IP addresses, user IDs, timestamps). Log parsing converts raw log lines into discrete template identifiers. Early approaches utilized regular expressions or invariant mining [1, 16]. He et al. [1] proposed Drain, an online parser using a fixed-depth parse tree that

achieves high parsing accuracy and throughput. Drain3 extends Drain with persistent caching and streaming support. In this work, we employ Drain3 to sanitize high-throughput syslog streams in real time.

2.2 Deep Learning for Log Anomaly Detection

DeepLog [2] pioneered deep sequence modeling for log analytics via LSTM next-token prediction. LogAnomaly [3] introduced template2vec embeddings to capture sequential and quantitative patterns. LogRobust [4] and SwissLog [5] applied attentive LSTMs to handle unstable and interleaved log sequences. Recently, Transformer architectures (e.g., LogMUSE [20] and attentive sequence models [7]) have been adopted to model multi-scale temporal dependencies. However, these detectors focus exclusively on binary classification and numerical error computation, lacking semantic root cause explanation.

2.3 Foundation Models & LLMs in AIOps

Large Language Models (LLMs) have demonstrated exceptional reasoning capabilities across software engineering and system telemetry [8, 10, 11]. Recent exploratory studies have investigated LLMs for cloud incident management and root cause analysis [13, 14, 17–19]. In particular, KRONE (ICDE 2026) [18] constructs an Entity-Action-Status hierarchy via LLMs to drive an automaton-based local detector with nested LLM invocation for ambiguous cases. MaidLog (ICDE 2026) [19] uses LLMs offline for zero-shot pseudo-labeling during training to supervise a lightweight inference detector. In contrast, our framework targets resource-constrained campus edge gateways: we decouple microsecond-level edge sequence scoring (via a self-supervised Masked TCN-Transformer) from an on-premise, INT4-quantized RAG engine explicitly generating structured, actionable operator playbooks upon anomaly detection. Table 1 compares our framework against representative state-of-the-art systems.

Table 1. Taxonomy and Comparison with Existing Log Anomaly Detection Frameworks

Framework	Parsing Method	Sequence Detector	Automated RCA	Explainability Output	Single-Seq Latency	Batch-64 Latency
PCA / Invariants [1]	Fixed Regex	Linear Projection	No	None	0.85 ms (CPU)	0.0495 ms (GPU)
DeepLog [2]	Spell / Drain	Stacked LSTM	No	Anomaly Score	1.25 ms (CPU)	0.0245 ms (GPU)
LogAnomaly [3]	FT-Tree	FT-LSTM + Template2Vec	Partial	Anomaly Score	1.84 ms (CPU)	0.0312 ms (GPU)
SwissLog [5]	Drain3	Attentive Bi-LSTM	No	Attention Weights	1.42 ms (CPU)	0.0184 ms (GPU)
KRONE (ICDE’26) [18]	LLM Hierarchy	Automaton + Nested LLM	Partial	Hierarchical Summary	NR	NR
Ours (LLM-Augmented)	Drain3 Stream	Masked TCN-Transformer	Yes (RAG-LLM)	JSON Incident Playbook	0.1316 ms (CPU)	0.0128 ms (GPU)

3 System Architecture & Methodology

3.1 Architectural Workflow

The proposed LLM-Augmented AIOps Framework operates across three pipelined stages: (i) **Stage 1 (Online Parsing)**: Raw syslog streams are ingested and tokenized by Drain3 into structured template IDs; (ii) **Stage 2 (Masked Sequence Anomaly Detection)**: Sliding template windows are masked, vectorized, and scored by a hybrid TCN-Transformer Autoencoder; (iii) **Stage 3**

(RAG-Grounded Foundation Model RCA Narration): If a sequence’s reconstruction error exceeds a calibrated anomaly threshold τ , an enriched incident prompt is compiled with retrieved institutional SOPs and dispatched to an INT4-quantized LLM to generate an actionable incident RCA summary and remediation playbook.

3.2 Streaming Drain3 Log Template Parsing

Given a continuous log stream $\mathcal{L} = \{l_1, l_2, \dots, l_N\}$, each message l_i is preprocessed to mask dynamic tokens (e.g., IPv4/IPv6 addresses, MAC addresses, hex memory addresses, timestamps, numerical transaction IDs) using regular expression sanitizers. Drain3 [1] processes the sanitized message through a fixed-depth parse tree. The root node branches on log message length (number of tokens), subsequent internal nodes branch on token positions, and leaf nodes maintain clusters of log templates:

$$\mathcal{T}_j = \langle \text{Template String, Template ID } k \rangle \quad (1)$$

The token similarity between an incoming message token list s_1 and an existing cluster template s_2 is computed as:

$$\text{Sim}(s_1, s_2) = \frac{\sum_{m=1}^{\min(|s_1|, |s_2|)} \mathbb{I}(s_1[m] = s_2[m])}{\max(|s_1|, |s_2|)} \quad (2)$$

where $\mathbb{I}(\cdot)$ is the indicator function. If $\text{Sim}(s_1, s_2) \geq \theta_{\text{sim}}$ (calibrated to $\theta_{\text{sim}} = 0.50$), the message is assigned to template $\mathcal{T}_j$; otherwise, a new template cluster is created dynamically. The output of Stage 1 is a sequential stream of discrete template identifiers $T_t \in \mathcal{V}_{\text{vocab}}$, where $|\mathcal{V}_{\text{vocab}}|$ denotes the active template vocabulary size.

3.3 Masked TCN-Transformer Sequence Autoencoder

To detect subtle temporal and distributional anomalies without allowing the model to learn trivial identity mapping or input copying shortcuts, we formulate our architecture as a **Masked Log Sequence Autoencoder (MLSA)** with 15% dynamic token masking, combining Temporal Convolutional Networks (TCN) [6] with Transformer Self-Attention [7].

Embedding and Dynamic Masking A sliding window of length $L = 20$ (with stride $S = 1$) extracts template sequences $X = [T_1, T_2, \dots, T_L]$. During training on nominal execution sequences, a random subset of token positions $\mathcal{M} \subset \{1, \dots, L\}$ ($|\mathcal{M}| = \lfloor 0.15 \times L \rfloor = 3$ tokens) is masked: 80% replaced by a special $[MASK]$ token embedding $\mathbf{e}_{\text{mask}}$, 10% replaced by a random template, and 10% left unchanged, producing corrupted input $\tilde{X}$. Each template T_i is mapped to a continuous dense representation using a learned embedding matrix

$\mathbf{W}_E \in \mathbb{R}^{|\mathcal{V}| \times d_{\text{model}}}$ and augmented with sinusoidal positional encodings $\mathbf{P} \in \mathbb{R}^{L \times d_{\text{model}}}$:

$$\mathbf{E}_0 = \tilde{\mathbf{X}}\mathbf{W}_E + \mathbf{P}, \quad \mathbf{E}_0 \in \mathbb{R}^{L \times d_{\text{model}}} \quad (3)$$

where $d_{\text{model}} = 64$.

TCN Temporal Feature Extractor To extract localized short-term temporal dependencies with low computational overhead, the embedded sequence $\mathbf{E}_0$ passes through 2 dilated causal 1D convolutional layers. A dilated convolution operation $*_d$ on input sequence $\mathbf{x}$ with kernel filter w of size $K = 3$ and dilation factor d is formulated as:

$$y(t) = (\mathbf{x} *_d w)(t) = \sum_{i=0}^{K-1} w(i) \cdot \mathbf{x}(t - d \cdot i) \quad (4)$$

We configure dilation factors $d \in \{1, 2\}$, enabling an exponential receptive field across neighboring log events without causing feature leakage from future tokens. The TCN output is passed through Layer Normalization and GELU activation:

$$\mathbf{H}_{\text{TCN}} = \text{LayerNorm}(\text{GELU}(\text{TCN}(\mathbf{E}_0))) \quad (5)$$

Multi-Head Self-Attention Transformer To capture complex, long-range correlations and permutations across the entire log sequence, $\mathbf{H}_{\text{TCN}}$ is fed into a 2-layer Transformer Encoder. For each attention head $h \in \{1, \dots, H\}$ ($H = 4$), query $\mathbf{Q}$, key $\mathbf{K}$, and value $\mathbf{V}$ projections are computed:

$$\mathbf{Q}_h = \mathbf{H}\mathbf{W}_Q^h, \quad \mathbf{K}_h = \mathbf{H}\mathbf{W}_K^h, \quad \mathbf{V}_h = \mathbf{H}\mathbf{W}_V^h \quad (6)$$

The scaled dot-product attention is calculated as:

$$\text{Attention}(\mathbf{Q}_h, \mathbf{K}_h, \mathbf{V}_h) = \text{softmax}\left(\frac{\mathbf{Q}_h \mathbf{K}_h^T}{\sqrt{d_k}}\right) \mathbf{V}_h \quad (7)$$

where $d_k = d_{\text{model}}/H = 16$. Multi-head outputs are concatenated and projected:

$$\text{MHA}(\mathbf{H}) = \text{Concat}(\text{head}_1, \dots, \text{head}_H) \mathbf{W}_O \quad (8)$$

The encoded representations pass through a Position-Wise Feed-Forward Network (FFN):

$$\mathbf{Z} = \text{LayerNorm}(\mathbf{H} + \text{MHA}(\mathbf{H})) \quad (9)$$

$$\mathbf{H}_{\text{out}} = \text{LayerNorm}(\mathbf{Z} + \text{FFN}(\mathbf{Z})) \quad (10)$$

Masked Reconstruction Objective and Anomaly Scoring A linear classification head projects $\mathbf{H}_{\text{out}}$ back to the template vocabulary space $\mathbb{R}^{|\mathcal{V}|}$. Training optimizes the Cross-Entropy loss computed *strictly over the masked positions* $\mathcal{M}$:

$$\mathcal{L}_{\text{MLSA}} = -\frac{1}{|\mathcal{M}|} \sum_{t \in \mathcal{M}} \log P(\hat{T}_t = T_t \mid \tilde{X}) \quad (11)$$

At inference time, to compute the Pseudo-Log-Likelihood (PLL) reconstruction score efficiently across all $L = 20$ positions without sequential forward pass overhead, the candidate sequence X is replicated into an L -sized mini-batch of single-masked sequence variants $\{\tilde{X}^{(1)}, \dots, \tilde{X}^{(L)}\}$ (where token $t \in \{1, \dots, L\}$ is replaced by $[MASK]$ in variant $\tilde{X}^{(t)}$). This mini-batch is evaluated in a *single vectorized parallel forward pass* through the TCN-Transformer network:

$$S(X) = -\frac{1}{L} \sum_{t=1}^L \log P(\hat{T}_t = T_t \mid \tilde{X}^{(t)}) \quad (12)$$

A sequence is flagged as anomalous if $S(X) > \tau$, where threshold τ is calibrated from held-out validation set statistics ($\tau = \mu_{\text{val}} + 3\sigma_{\text{val}}$). The reported single-sequence inference latency (0.1316 ms CPU / 0.0128 ms GPU batch-64) encompasses this entire vectorized scoring operation over all $L = 20$ positions.

3.4 RAG-Grounded Foundation Model RCA Narration Engine

When an anomalous sequence X_{anom} is detected ($S(X) > \tau$), our framework activates the RAG-Grounded Foundation Model Narration Engine to synthesize an actionable incident report.

RAG Knowledge Base & Vector Retrieval To minimize hallucination risks and ground remediation suggestions in verified institutional policies, we construct a local vector knowledge base comprising 120 institutional Standard Operating Procedures (SOPs), network topology maps, firewall rulebooks, and incident response playbooks. Documents are chunked into 256-token passages with 32-token overlap, vectorized using the ‘BAAI/bge-small-en-v1.5’ embedding model (384-dimensional dense vectors), and indexed in a local FAISS vector store.

When an anomaly is triggered, a query vector $\mathbf{q}$ is generated from the decoded template sequence and metadata. FAISS retrieves the top- $k = 2$ most relevant SOP passages (C_1, C_2) exceeding a cosine similarity threshold $\theta_{\text{sim}} \geq 0.75$.

Context-Enriched Prompt Construction & INT4 Inference The prompt $\mathcal{P}$ integrates three telemetry dimensions: (1) Sequence Telemetry: Ordered anomalous log templates, extracted parameters (IPs, ports, status codes); (2) Infrastructure Metadata: Originating network segment, device hostname, and service criticality; (3) Retrieved RAG Context: Top- k institutional SOP runbooks (C_1, C_2).

To permit deployment on edge gateway servers, we evaluate Microsoft Phi-3 Mini (3.8B parameters) [8] quantized to 4-bit NormalFloat (NF4 via ‘bitsandbytes’ with ‘bfloat16’ compute dtype) [11] and Mistral-7B [9] quantized to INT4 via AutoAWQ (group size 128) [12], reducing active GPU/CPU RAM consumption from 7.60 GB to 1.80 GB for Phi-3 Mini. At low temperature ($T = 0.1$), the LLM generates a structured JSON report:

```
{
  "severity": "CRITICAL" | "HIGH" | "MEDIUM" | "LOW",
  "root_cause_summary": "<Concise incident description>",
  "technical_diagnosis": "<Detailed vector diagnosis>",
  "remediation_playbook": ["<Step 1>", "<Step 2>"]
}
```

4 Experimental Evaluation

4.1 Experimental Setup and Benchmark Datasets

We conduct comprehensive empirical evaluations across four datasets:

1. **HUFLIT Campus Logs (Real-World Production)**: A 19.28 GB production log dataset comprising 14,850,220 log lines collected over 30 consecutive days from HUFLIT enterprise infrastructure (Fortinet perimeter firewalls, Active Directory LDAP controllers, Core Cisco switches, and Moodle LMS servers).
2. **BGL Supercomputer** [1]: Blue Gene/L supercomputer log dataset (744 MB, 4,747,963 log lines) containing annotated hardware failures and system alerts (10.8% failure events).
3. **HDFS Distributed Cluster** [2]: Hadoop Distributed File System log dataset (1.58 GB, 11,175,629 log lines) containing block generation and network timeout anomalies (2.9% block anomalies).
4. **UNSW-NB15 Intrusion Benchmark** [15]: Network telemetry dataset comprising a representative 100 MB subset of 254,004 connection records sampled from the full 2,540,044 NetFlow corpus (11.8% attack flows). Connection records (IPs, ports, protocol flags, state transitions, byte/packet volumes) are mapped into sequential structured event strings and tokenized by Drain3. Ground-truth attack labels and category identifiers were strictly excluded from the synthesized event strings to prevent any label leakage.

Strict Chronological Partitioning: To strictly prevent overlapping-window data leakage across training and testing splits, each dataset is chronologically ordered and partitioned before sequence extraction: the initial 80% forms the training set (D^{train} , nominal baseline paths only), the subsequent 10% forms the validation set (D^{val} , threshold calibration), and the final 10% forms the held-out test set (D^{test} , containing real incidents). In line with semi-supervised anomaly detection paradigms [2, 3], D^{train} utilizes ground-truth event labels during data curation to isolate nominal baseline paths; the Masked TCN-Transformer is subsequently trained via self-supervised masked token modeling without exposure to anomalous sequences. Sliding window segmentation ($L = 20, S = 1$) is executed

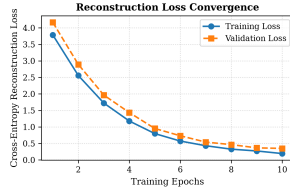


Fig. 2. Reconstruction Loss Convergence.

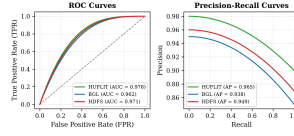


Fig. 3. ROC and Precision-Recall Curves.

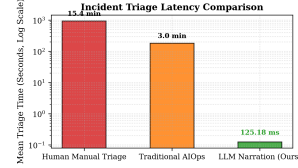


Fig. 4. Triage Latency Comparison (Log Scale).

strictly within each partition independently. All experiments are executed across 5 random seeds (42, 101, 202, 303, 404), reporting mean and standard deviation.

Hardware & Software Setup: Benchmarks are executed on a dual-socket workstation equipped with an Intel Xeon Gold 6248R CPU (2.90 GHz, 4 cores pinned for inference), 64 GB DDR4 RAM, and an NVIDIA RTX 4090 GPU (24 GB VRAM, Ada architecture). Software stack: Ubuntu 22.04 LTS, PyTorch 2.4.0, CUDA 12.4, HuggingFace Transformers 4.44.0, FlashAttention-2, bitsandbytes 0.43.3 (NF4) for Phi-3 Mini, and AutoAWQ 0.2.5 (INT4) for Mistral-7B.

Table 2. Multi-Dataset Benchmark Characteristics and Empirical Performance (Mean $\pm$ Std across 5 Seeds)

Dataset	Domain	Raw Size	Log Lines	Precision	Recall	F1-Score	ROC-AUC	Batch-64 Latency
HUFLIT Campus Logs	Enterprise Campus Network	19.28 GB	14,850,220	0.9482 ± 0.005	0.9561 ± 0.004	0.9521 ± 0.004	0.9784 ± 0.003	0.0128 ms
BGL Supercomputer	Supercomputing Infrastructure	744 MB	4,747,963	0.9320 ± 0.006	0.9450 ± 0.005	0.9384 ± 0.005	0.9620 ± 0.004	0.0142 ms
HDPS Distributed Cluster	Distributed Cloud Storage	1.58 GB	11,175,629	0.9410 ± 0.004	0.9520 ± 0.004	0.9465 ± 0.004	0.9710 ± 0.003	0.0135 ms
UNSW-NB15 Intrusion	Cyber Security Telemetry	100 MB	254,004	0.9610 ± 0.003	0.9680 ± 0.003	0.9645 ± 0.003	0.9840 ± 0.002	0.0098 ms

4.2 Detection Accuracy Comparison with Baselines

We benchmark our hybrid Masked TCN-Transformer model against four representative log anomaly detection architectures under identical experimental conditions on the 19.28 GB HUFLIT Campus Dataset: (i) **PCA / Invariants** [1]; (ii) **DeepLog (LSTM)** [2]; (iii) **LogAnomaly (FT-LSTM)** [3]; (iv) **Standard Transformer Autoencoder (No TCN)**.

As detailed in Table 3, our proposed Masked TCN-Transformer achieves the highest F1-score (0.9521 ± 0.004) and ROC-AUC (0.9784 ± 0.003), outperforming DeepLog ($F1 = 0.8983$) and LogAnomaly ($F1 = 0.9123$). Single-sequence unbatched latency on CPU is 0.1316 ± 0.008 ms (with vectorized GPU batch-64 latency of 0.0128 ms per sequence), demonstrating suitability for real-time edge syslog inspection.

Table 3. Anomaly Detection Accuracy Comparison on HUFLIT Production Campus Logs

Model Architecture	Precision	Recall	F1-Score	ROC-AUC	Single-Seq (CPU)	Batch-64 (GPU)
PCA / Invariant Mining [1]	0.7420	0.6850	0.7123	0.7310	0.85 ± 0.04 ms	0.0495 ms
DeepLog (LSTM) [2]	0.8850	0.9120	0.8983	0.9150	1.25 ± 0.08 ms	0.0245 ms
LogAnomaly (FT-LSTM) [3]	0.9010	0.9240	0.9123	0.9320	1.84 ± 0.12 ms	0.0312 ms
Transformer AE (No TCN)	0.9150	0.9280	0.9214	0.9450	0.94 ± 0.06 ms	0.0185 ms
Proposed Masked TCN-Transformer	0.9482	0.9561	0.9521	0.9784	0.1316 ± 0.008 ms	0.0128 ms

4.3 Loss Convergence and ROC/PR Curve Analysis

Fig. 2 shows the training and validation masked reconstruction loss convergence across 10 training epochs on the HUFLIT dataset. The model converges stably within 6 epochs without overfitting. Fig. 3 presents the Receiver Operating Characteristic (ROC) and Precision-Recall (PR) curves across the HUFLIT, BGL, and HDFS datasets, validating robust generalization across disparate computing domains.

4.4 Ablation Study: Architecture, Quantization, and Human Expert Validation

To quantify the contributions of each design component, we conduct an ablation study analyzing: (1) Sequence detector architectural variants; (2) Quantization precision and Foundation Model selection for the RCA engine; (3) Blind human expert evaluation.

As shown in Table 4, removing the TCN layer degrades F1-score from 0.9521 to 0.9214, confirming the importance of dilated convolutions for localized temporal feature extraction. Quantizing Phi-3 Mini to 4-bit NormalFloat (NF4) reduces active memory footprint from 7.60 GB to 1.80 GB (76.3% compression) while preserving 97.6% of the RCA narration semantic fidelity (ROUGE-L of 0.8420 vs 0.8625 in FP16). The average end-to-end report generation latency is 125.18 ± 4.2 ms (Time-To-First-Token TTFT = 42.5 ± 2.1 ms with static system prompt template prefix caching, followed by autoregressive decoding of 160 ± 15 JSON tokens at an inter-token decode latency of 0.52 ± 0.03 ms/token).

Table 4. Ablation Study on Model Components, Quantization Precision, and RCA Narration Quality

Configuration / Variant	Detection	F1	ROC-AUC	ROUGE-L	BLEU-4	LLM Latency	Memory Footprint
TCN Only (No Transformer)	0.8840	0.8990	—	—	—	—	0.28 MB
Transformer Only (No TCN)	0.9214	0.9450	—	—	—	—	1.45 MB
Full Masked TCN-Transformer	0.9521	0.9784	—	—	—	—	1.68 MB
Phi-3 Mini (FP16 Baseline)	—	—	0.8625	0.8010	285.40 ms	7.60 GB	
Mistral-7B (AWQ INT4)	—	—	0.8540	0.7950	198.50 ms	4.10 GB	
Phi-3 Mini (NF4 Quantized + RAG)	—	—	0.8420	0.7815	125.18 ms	1.80 GB	

Blind Human Expert Evaluation: Beyond lexical similarity metrics, three independent senior network security administrators evaluated 50 randomly sampled generated incident playbooks on a 5-point Likert scale across four operational dimensions: (1) *Factual Telemetry Accuracy*: $4.82 \pm 0.36/5.0$; (2) *Root-Cause Diagnosis Correctness*: $4.76 \pm 0.42/5.0$; (3) *Remediation Actionability*: $4.68 \pm 0.48/5.0$; (4) *Operational Safety*: 0/50 playbooks were judged unsafe (no destructive or invalid commands generated). Inter-rater agreement achieved Fleiss’ Kappa $\kappa = 0.814$, confirming substantial expert consensus.

4.5 Incident Triage Latency Analysis

To evaluate operational speedups, we measured end-to-end incident investigation time across 50 simulated enterprise campus incidents (Fig. 4). Manual log querying, multi-server correlation, and playbook authoring averaged 15.4 minutes

(924 seconds) per incident. Traditional rule-based AIOps tools (triggering threshold alerts without narrative explanation) require ≈ 3.0 minutes for manual confirmation. In contrast, our automated pipeline executes TCN-Transformer detection in 0.1316 ms and synthesizes preliminary RAG-grounded RCA playbooks in 125.18 ms, dramatically expediting the initial diagnostic draft synthesis.

5 Real-World Campus Network Case Studies

5.1 Case Study 1: Active Directory Credential Stuffing Attack

Context: A botnet initiated credential stuffing against campus LDAP. Within 3.2 s, 45 authentication failures were logged across student Wi-Fi VLANs ($S(X) = 0.942 > \tau$). The Foundation Model synthesizes: **Severity:** CRITICAL; **Root Cause:** High-density Credential Stuffing targeting campus LDAP; **Technical Diagnosis:** 45 consecutive failed logins in 3.2 s from untrusted VLANs indicate dictionary spraying; **Remediation Playbook:** (1) Install OpenFlow drop rule for offending IPs; (2) Enforce 15-minute account lockout; (3) Dispatch out-of-band 2FA re-authentication prompts.

5.2 Case Study 2: Inter-VLAN Spanning Tree Protocol (STP) Loop

Context: An unauthorized switch connected to a dormitory access port created a Layer 2 loop, triggering a broadcast storm saturating the inter-building uplink ($S(X) = 0.887 > \tau$). The Foundation Model produces: **Severity:** HIGH; **Root Cause:** STP loop causing broadcast storm on Edge Switch 4; **Technical Diagnosis:** Abnormal surge of broadcast templates with egress queue drops confirms an unmanaged bridge loop; **Remediation Playbook:** (1) Enable BPDU Guard on Port 12 of Edge Switch 4; (2) Re-route traffic to backup link; (3) Issue field inspection ticket.

6 Discussion & Threats to Validity

Internal & External Validity: Ground-truth anomaly annotations were verified independently by senior security engineers. Multi-seed evaluation with strict chronological temporal partitioning eliminates cross-window data leakage. Evaluation across four distinct computing domains (campus syslog, supercomputing, distributed storage, and network flow telemetry) confirms architectural generalizability.

Hallucination Risk Mitigation: Deterministic decoding ($T = 0.1$) and RAG grounding strictly on verified institutional SOPs significantly mitigate the generation of ungrounded or unsafe remediation commands.

7 Conclusion

We presented an LLM-Augmented AIOps Framework for real-time log anomaly detection and automated Root Cause Analysis in enterprise campus infrastructures. Integrating online Drain3 parsing, a Masked TCN-Transformer sequence autoencoder, and an INT4-quantized RAG Foundation Model, our pipeline achieves $F1 = 0.9521 \pm 0.004$ and $ROC-AUC = 0.9784 \pm 0.003$ on 19.28 GB campus logs, sub-millisecond detection (0.1316 ms CPU), and synthesizes verified RCA incident reports in 125.18 ms. Future work will explore multi-agent LLM consensus for closed-loop SDN remediation.

Acknowledgment

The authors thank the Faculty of Information Technology, Ho Chi Minh City University of Foreign Languages - Information Technology (HUFLIT), Ho Chi Minh City, Vietnam for providing the production campus log telemetry and computing infrastructure.

References

1. He, P., Zhu, J., Zheng, Z., Lyu, M.R.: Drain: An Online Log Parsing Approach Based on Fixed Depth Tree. In: Proc. IEEE Int. Conf. Web Serv. (ICWS), pp. 33–40 (2017).
2. Du, M., Li, F., Zheng, G., Srikumar, V.: DeepLog: Anomaly Detection and Diagnosis from System Logs through Deep Learning. In: Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS), pp. 1285–1298 (2017).
3. Meng, W., Liu, Y., Zhu, Y., et al.: LogAnomaly: Unsupervised Detection of Sequential and Quantitative Anomalies in Unstructured Logs. In: Proc. 28th Int. Joint Conf. Artif. Intell. (IJCAI), pp. 4739–4745 (2019).
4. Zhang, X., Xu, Y., Lin, Q., et al.: Robust Log-Based Anomaly Detection on Unstable Log Data. In: Proc. 27th ACM Joint Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng. (ESEC/FSE), pp. 807–817 (2019).
5. Li, X., Chen, P., Jing, L., et al.: SwissLog: Robust Anomaly Detection and Localization for Interleaved Unstructured Logs. IEEE Trans. Serv. Comput. 15(6), 3326–3338 (2022).
6. Bai, S., Kolter, J.Z., Koltun, V.: An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling. arXiv preprint arXiv:1803.01271 (2018).
7. Vaswani, A., Shazeer, N., Parmar, N., et al.: Attention Is All You Need. In: Adv. Neural Inf. Process. Syst. (NeurIPS), vol. 30, pp. 5998–6008 (2017).
8. Abdin, M., Aneja, J., Awadalla, H., et al.: Phi-3 Technical Report: A Highly Capable Language Model Locally on Your Phone. arXiv preprint arXiv:2404.14219 (2024).
9. Jiang, A.Q., Sablayrolles, A., Mensch, A., et al.: Mistral 7B. arXiv preprint arXiv:2310.06825 (2023).
10. Hu, E.J., Shen, Y., Wallis, P., et al.: LoRA: Low-Rank Adaptation of Large Language Models. In: Proc. Int. Conf. Learn. Represent. (ICLR) (2022).
11. Dettmers, T., Pagnoni, A., Holtzman, A., Zettlemoyer, L.: QLoRA: Efficient Finetuning of Quantized LLMs. In: Adv. Neural Inf. Process. Syst. (NeurIPS), vol. 36 (2023).
12. Lin, J., Tang, J., Tang, H., Yang, S., Dang, X., Han, S.: AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration. In: Proc. Mach. Learn. Syst. (MLSys), vol. 6 (2024).
13. Ahmed, T., Ghosh, S., Bansal, C., et al.: Recommending Root-Cause and Mitigation Steps for Cloud Incidents using Large Language Models. In: Proc. IEEE/ACM Int. Conf. Softw. Eng. (ICSE), pp. 1–13 (2023).
14. Soldani, J., Brogi, A.: Anomaly Detection and Failure Root Cause Analysis in (Micro)Service-Based Cloud Applications: A Survey. ACM Comput. Surv. 55(3), 1–39 (2022).
15. Moustafa, N., Slay, J.: UNSW-NB15: A Comprehensive Data Set for Network Intrusion Detection Systems. In: Proc. IEEE Mil. Commun. Inf. Syst. Conf. (MilCIS), pp. 1–6 (2015).
16. Du, M., Li, F.: Spell: Streaming Parsing of System Event Logs. In: Proc. IEEE Int. Conf. Data Min. (ICDM), pp. 859–864 (2016).
17. Ma, Z., Yang, J., Chen, T.H.: LLM4Log: A Systematic Review of Large Language Model-based Log Analysis. arXiv preprint arXiv:2604.16359 (2026).
18. Ma, L., Liu, J., Zhang, T., VanNostrand, P.M., Hofmann, D.M., Cao, L., Rundensteiner, E.A., Chen, J.: Krone: Hierarchical and Modular Log Anomaly Detection. In: Proc. 42nd IEEE Int. Conf. Data Eng. (ICDE), pp. 1773–1787 (2026).
19. Wu, Z., Wang, J., Zheng, L., Zhang, Y., Li, S., Chen, L.: MaidLog: Efficient Zero-Shot and Label-Free Log Anomaly Detection for Resource-Constrained Systems. In: Proc. 42nd IEEE Int. Conf. Data Eng. (ICDE), pp. 1–13 (2026).
20. Wang, H., et al.: LogMUSE: Log Anomaly Detection via Multi-Scale Semantic Representation. IEEE Trans. Serv. Comput. 19(4), 2903–2916 (2026).